

Egy egyszerű és hatékony algoritmus egy minimális feszítőfa helyettesítő éleinek megtalálására

David A. Bader, Paul Burkhardt

<http://jgaa.info/> vol. 26, no. 4, pp. 577-588 (2022)

Tóth Ábel Tibor, Nagy Bence

2025. november 17.

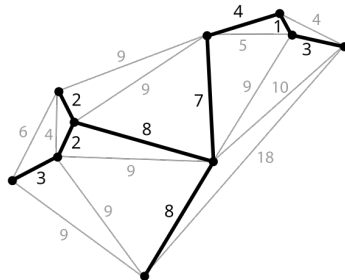
Áttekintés: minimális feszítőfa, csereél keresése

Definition

Legyen $G = (V, E)$ egy irányítatlan, súlyozott gráf, ahol $n = |V|$ és $m = |E|$, és minden $e \in E$ élhez tartozik egy $w(e)$ súly. Egy minimális feszítőfa $T = MST(G)$ a gráf olyan $n - 1$ élből álló részhalmaza, amely összeköti az összes csúcsot, és az élek összsúlya minimális.

Definition

A csereél az a legkisebb súlyú él, amely újra összeköti a minimális feszítőfát (egy minimális feszítőfabeli él törlése után).



Kép forrás: Wikipédia

Az algoritmus célja és eredménye

- **Cél:** keressük meg a minimális költségű csereélét minden minimális feszítőfabeli élnek.
- A bemutatott algoritmus képes megtalálni ezeket, ha a nem min feszfabeli élek élsúly szerint növekvő sorrendben rendezve megvannak határozva:
 - **Idő- és tárbonyolultság:** $\mathcal{O}(m + n)$
- Megjegyzés: a nem min. feszfabeli éleket például a Kruskal-algoritmus használatakor rendezve kapjuk.
- A cikk előtti legjobb eredmény egy $\mathcal{O}(m\alpha(m, n))$ időbonyolultságú algoritmus volt Tarjan-tól 1979-ből. (megj.: $\alpha(m, n)$: Inverse Ackermann's function)

Algoritmus bemutatása: PathLabel

```
procedure PathLabel( $s, t, e$ )
2: if  $IN[s] < IN[t] < OUT[s]$  then                                ▷  $s$  is ancestor of  $t$ 
3:   return
4: if  $IN[t] < IN[s] < OUT[t]$  then                                ▷  $t$  is ancestor of  $s$ 
5:   PLAN  $\leftarrow$  ANCESTOR;  $k_1 \leftarrow IN[t]$ ;  $k_2 \leftarrow IN[s]$ 
6: else
7:   if  $IN[s] < IN[t]$  then
8:     PLAN  $\leftarrow$  LEFT;  $k_1 \leftarrow OUT[s]$ ;  $k_2 \leftarrow IN[t]$           ▷  $s$  is left of  $t$ 
9:   else
10:    PLAN  $\leftarrow$  RIGHT;  $k_1 \leftarrow OUT[t]$ ;  $k_2 \leftarrow IN[s]$           ▷  $s$  is right of  $t$ 
11:   $v \leftarrow s$ 
12:  while  $k_1 < k_2$  do                                           ▷ Detecting when below LCA( $s, t$ )
13:    if find( $v$ ) =  $v$  then                                         ▷ If true, set replacement edge for ( $v, P[v]$ )
14:       $R(v, P[v]) \leftarrow e$                                    ▷ Set the replacement edge
15:      link( $v$ )
16:       $v \leftarrow$  find( $v$ )                                       ▷ Union the disjoint sets of  $v$  and  $P[v]$ 
17:    switch PLAN do
18:      case ANCESTOR
19:         $k_2 \leftarrow IN[v]$ 
20:      case LEFT
21:         $k_1 \leftarrow OUT[v]$ 
22:      case RIGHT
23:         $k_2 \leftarrow IN[v]$ 
```

Algoritmus: Initialization

```
24: Root the MST  $T$  at arbitrary vertex  $v_r$  and store parents in  $P$  .
25:  $P[v_r] \leftarrow v_r$  ▷ root's parent points to root
26: Run DFS on  $T$ , setting  $IN[v]$  and  $OUT[v]$  to the counter value when  $v$  is first and last visited, respectively.
27: for all vertices  $v$  in  $V$  do
28:    $make\_set(v)$  ▷ Initialize the disjoint sets
29: for all edges  $e$  in  $E_T$  do
30:    $R_e = \text{empty}$  ▷ Initialize the replacement edges
31: for  $k \leftarrow 1 \dots m - n + 1$  do ▷ Scan the  $m - n + 1$  sorted non-MST edges
32:    $(v_i, v_j) \leftarrow e_k$ 
33:    $PathLabel(v_i, v_j, (v_i, v_j))$ 
34:    $PathLabel(v_j, v_i, (v_i, v_j))$ 
```

Legfontosabb él (Most Vital Edge)

- **Definíció:** Az az él, amelynek eltávolítása a legnagyobb növekedést okozza az MST (minimális feszítőfa) súlyában.
- **Megjegyzés:** Ha G -ben van híd, a legfontosabb él nincs értelmezve.
- **Kulcsállítás:** A legfontosabb él az MST-ben van.
- **Cikk hozzájárulása:** Ha adott az MST és a nem-fa élek súly szerint rendezve:
 - 1 Minden csereél meghatározható $\mathcal{O}(m+n)$ időben.
 - 2 A legfontosabb él kiválasztható $\mathcal{O}(n)$ időben